

SIMATS ENGINEERING
CO3 - ASSESSMENT TOOL 3

Comparative Analysis

COURSE NAME: DEEP LEARNING

COURSE CODE: MLA04

COURSE OUTCOME COVERED

CO3: Students will be able to understand, analyze and apply CNN concepts including layers, filters, parameter sharing, regularization, AlexNet and ResNet for real-world image processing applications. (BTL-6)

CO Weightage: 25%

Assessment Tool Weightage: 40%

QUESTIONS:

Total Marks:25

Q1. CNN vs Fully Connected Neural Network

A medical imaging organization wants to classify X-ray images using deep learning. Compare a **CNN with a traditional fully connected neural network** in terms of architectural design, motivation, spatial feature extraction, number of parameters, computational complexity, and ability to handle image data. Analyze the roles of convolutional layers, filters, and pooling layers, and justify why CNNs are more suitable for image classification applications.

Q2. Different CNN Layers and Filters

A manufacturing company is developing a CNN-based system to detect defects in industrial products. Compare the functions of **convolutional layers, pooling layers, activation layers, and fully connected layers** in terms of feature extraction, spatial information, computational requirements, and classification. Further compare **early-layer filters and deeper-layer filters** based on the type of features they learn, and justify how these layers collectively improve defect detection.

Q3. Parameter Sharing vs Independent Weights

A smart surveillance system needs to process thousands of high-resolution images efficiently. Compare **parameter sharing in CNNs with independent weights in fully connected networks** in terms of the number of parameters, memory requirements, computational complexity, feature detection, and translation-related invariance. Analyze how the reuse of convolutional filter weights improves learning efficiency and justify its importance for large-scale image-processing applications.

Q4. AlexNet vs ResNet

A computer vision company is developing an image classification system and is considering **AlexNet and ResNet** as possible architectures. Compare the two architectures in terms of network depth, layer organization, convolutional filters, parameter requirements, feature extraction capability, training difficulty, vanishing-gradient problem, computational cost, and classification performance. Recommend the more suitable architecture for a complex image recognition application and justify your choice.

Q5. Regularized CNN vs Non-Regularized CNN

An agricultural organization is developing a CNN to classify plant diseases from leaf images. The initial model achieves very high training accuracy but performs poorly on unseen images. Compare a **CNN without regularization** and a **CNN using dropout, data augmentation, and batch normalization** in terms of overfitting, generalization, training stability, computational requirements, and model performance. Analyze which regularization techniques should be applied and justify the most appropriate approach for reliable real-world disease classification.

Code of Conduct:

I T.chaitra(192425217) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: T.chaitra

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Selection of Studies/Parameters	20%	Selects highly relevant and diverse studies with well-defined comparison parameters	Selects relevant studies with minor gaps in parameter definition	Limited or partially relevant selection	Poor or inappropriate selection of studies
Comparison & Differentiation	25%	Clearly compares multiple aspects (methods, results, limitations) with strong differentiation	Good comparison with minor gaps	Basic comparison with limited depth	No clear comparison; mostly descriptive
Critical Evaluation	25%	Critically evaluates strengths, weaknesses, and implications with strong justification	Provides evaluation with some justification	Limited evaluation; lacks depth	No critical evaluation provided
Synthesis & Insight Generation	20%	Generates meaningful insights and conclusions from comparisons	Some insights derived from comparison	Limited or unclear insights	No insights generated
Clarity	10%	Information is clearly presented (tables/figures) with logical structure	Generally clear with minor issues	Presentation lacks clarity or structure	Poor presentation and difficult to understand

Q1. CNN vs Fully Connected Neural Network

A **Convolutional Neural Network (CNN)** is a deep learning architecture specially designed for processing image and spatial data. A traditional **Fully Connected Neural Network (FCNN)** connects every neuron in one layer to every neuron in the next layer. For X-ray image classification, CNNs are more suitable because they can automatically learn spatial and visual features while using significantly fewer parameters.

Comparison between CNN and Fully Connected Neural Network

Feature	CNN	Fully Connected Neural Network
Architecture	Uses convolution, activation, pooling and fully connected layers	Mainly consists of fully connected layers
Motivation	Designed to preserve and learn spatial relationships in images	Designed for general numerical/tabular data
Spatial feature extraction	Automatically extracts edges, textures, shapes and objects	Does not explicitly preserve spatial relationships
Parameters	Fewer parameters due to local connections and parameter sharing	Very large number of parameters
Memory requirement	Lower	Higher
Computational complexity	More efficient for image processing	Computationally expensive for high-resolution images
Image handling	Directly processes image structures	Image usually needs to be flattened
Translation handling	More robust to changes in object location	Sensitive to changes in pixel positions
Feature learning	Hierarchical feature learning	Learns features without spatial hierarchy
Suitability for X-rays	Highly suitable	Less suitable

Role of Convolutional Layers

Convolutional layers apply small filters or kernels to the input image. The filter moves across the image and performs mathematical operations to generate a **feature map**.

For example:

- First convolutional layers detect edges and simple patterns.
- Middle layers detect textures and shapes.
- Deeper layers detect complex structures such as organs or abnormal regions.

Role of Filters

A **filter** is a small matrix of learnable weights. Different filters learn different visual patterns.

For example:

- Vertical edge filter → detects vertical edges.
- Horizontal edge filter → detects horizontal edges.
- Texture filter → detects texture patterns.
- Deeper filters → detect complex medical structures.

Role of Pooling Layers

Pooling reduces the spatial dimensions of feature maps.

Common types are:

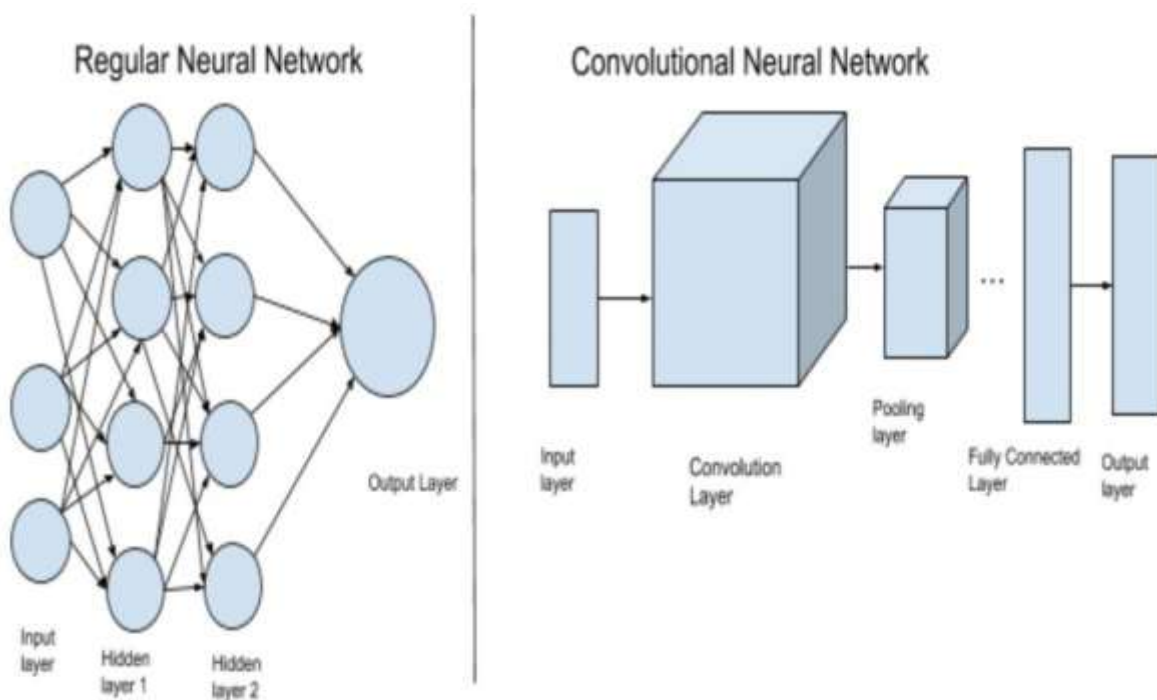
- **Max Pooling** – selects the maximum value from a region.
- **Average Pooling** – calculates the average value.

Pooling reduces computation, memory usage and sensitivity to small changes in object position.

Why CNN is Better for X-ray Classification

X-ray images contain important spatial information. Flattening an X-ray into a vector, as done in a traditional FCNN, loses the natural relationship between neighboring pixels. CNNs preserve these relationships and learn hierarchical features automatically.

Therefore, CNNs are more suitable for X-ray classification because they require fewer parameters, efficiently extract spatial features, reduce computational cost, and provide better image classification performance.



Q2. Different CNN Layers and Filters

A CNN consists of several types of layers that work together to extract useful features and perform classification. In an industrial defect detection system, these layers help identify scratches, cracks, dents, irregular shapes and other manufacturing defects.

Comparison of CNN Layers

Layer	Main Function	Spatial Information	Computational Requirement	Role
Convolutional Layer	Extracts features using filters	Preserves spatial information	Medium	Feature extraction
Pooling Layer	Reduces feature-map size	Reduces spatial dimensions	Low	Downsampling
Activation Layer	Introduces non-linearity	Maintains feature-map structure	Low	Enables complex learning
Fully Connected Layer	Combines extracted features	Spatial information mostly lost	High	Final classification

1. Convolutional Layer

The convolutional layer applies learnable filters to the input image. Each filter detects a particular pattern.

For defect detection, convolutional layers can identify:

- Edges
- Cracks
- Scratches
- Texture changes
- Surface irregularities
- Shapes of defective regions

The output is called a **feature map**.

2. Pooling Layer

Pooling reduces the size of feature maps while retaining important information.

For example, **Max Pooling** selects the maximum value from a small region.

Advantages:

- Reduces computational cost.
- Reduces memory requirements.
- Decreases feature-map dimensions.
- Provides some robustness to small translations.

3. Activation Layer

Activation functions introduce non-linearity into the network.

The most commonly used activation function in CNNs is **ReLU**:

$$\text{ReLU}(x) = \max(0, x)$$

ReLU allows the network to learn complex patterns instead of behaving like a simple linear model.

4. Fully Connected Layer

Fully connected layers are generally placed near the end of the CNN. They combine the high-level features extracted by previous layers and perform classification.

For example, the system can classify a product as:

- Normal
- Cracked
- Scratched
- Dented
- Defective

Early-Layer Filters vs Deeper-Layer Filters

Feature	Early-Layer Filters	Deeper-Layer Filters
Features learned	Simple features	Complex features
Examples	Edges, corners, colors	Shapes, textures, objects
Receptive field	Small	Large
Feature complexity	Low	High
Defect detection	Detect basic boundaries	Identify complete defect patterns

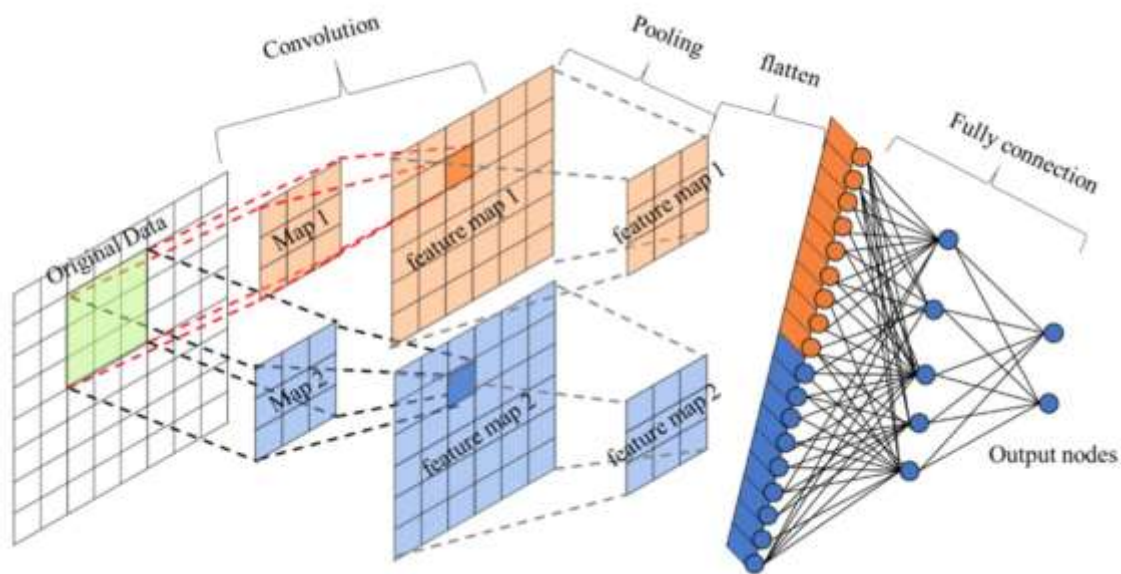
How the Layers Improve Defect Detection

The CNN works hierarchically:

Input Image → Edges → Textures → Shapes → Defect Patterns → Classification

For example, a crack may initially appear as simple edges. Deeper convolutional layers combine these edges into lines and shapes, allowing the network to recognize the complete crack pattern.

Therefore, convolutional layers extract features, pooling reduces unnecessary spatial information, activation functions enable nonlinear learning, and fully connected layers perform classification. Together, they provide an efficient solution for industrial defect detection.



Q3. Parameter Sharing vs Independent Weights

Parameter sharing is one of the most important characteristics of CNNs. In a CNN, the same convolutional filter is reused across different spatial locations of an image. In contrast, a fully connected network normally assigns separate weights to every input-output connection.

Comparison

Feature	CNN with Parameter Sharing	Fully Connected Network
Weights	Same filter reused across image	Independent weights
Number of parameters	Low	Very high
Memory requirement	Low	High
Computational cost	Lower for image feature extraction	Very high for high-resolution images
Feature detection	Same feature can be detected anywhere	Position-specific
Spatial structure	Preserved	Lost after flattening
Learning efficiency	High	Lower
Translation robustness	Better	Poorer
Large-scale images	Highly suitable	Less suitable

Parameter Sharing in CNN

Suppose a CNN uses a 3×3 **filter**. The same nine filter weights are applied repeatedly across different locations of the image.

For example:

Image → Filter → Feature Map

The filter can detect the same edge whether it appears on the left, center or right side of the image.

This is called **parameter sharing**.

Independent Weights in Fully Connected Networks

Consider a 224×224 RGB image.

Number of input values:

$$224 \times 224 \times 3 = 150,528$$

If this is connected to 1,000 neurons in a fully connected layer:

$$\text{Parameters} = 150,528 \times 1,000 = 150,528,000$$

This requires more than **150 million weights** for just one connection layer.

A CNN can use a small number of convolutional filters repeatedly, drastically reducing the number of parameters.

Memory Requirements

Because CNNs reuse the same filter weights, they require significantly less memory than fully connected networks. This is particularly important when processing thousands of high-resolution surveillance images.

Feature Detection

Parameter sharing allows a CNN to detect the same feature at different positions.

For example, a CNN trained to detect a person's face can recognize relevant patterns even if the person appears at different locations in the image.

Translation-Related Invariance

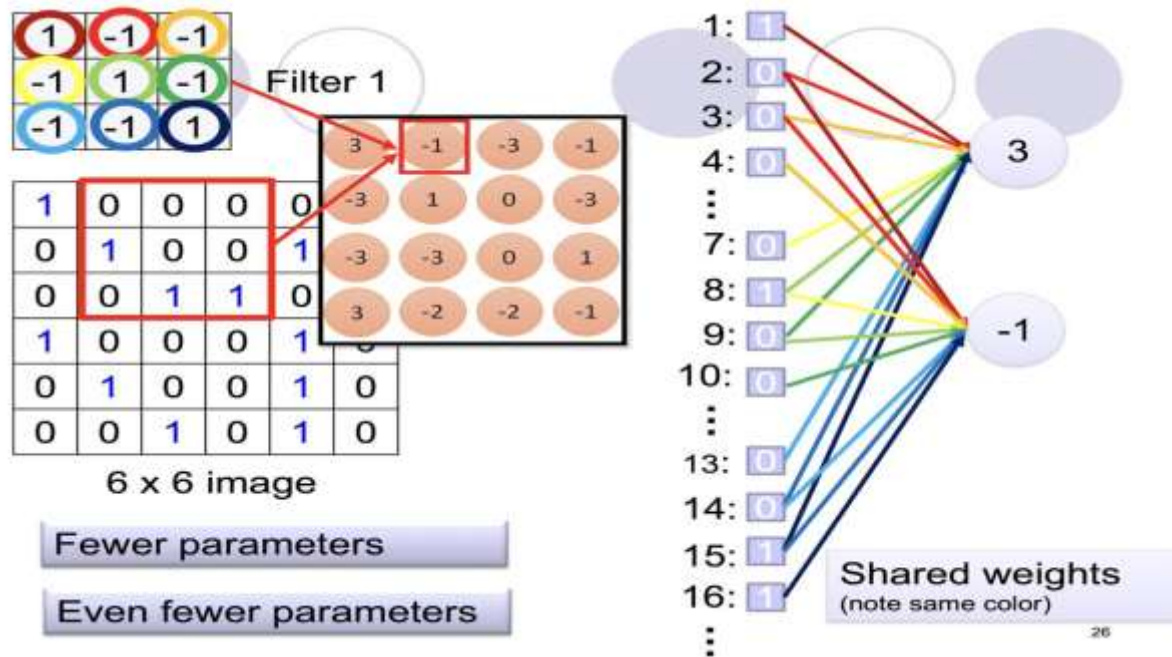
CNNs provide a degree of **translation robustness** because the same filter scans the entire image. Pooling further increases this robustness by reducing sensitivity to small spatial changes.

However, CNNs are not completely translation invariant by default; they are better described as having **translation-equivariant convolutional feature extraction with some invariance introduced through operations such as pooling and learned representations**.

Importance for Large-Scale Surveillance

A surveillance system may need to process thousands of high-resolution images. Independent weights would require huge memory and computation. Parameter sharing significantly reduces the model size while allowing the network to detect the same visual patterns at different locations.

Therefore, **parameter sharing is essential for efficient CNN learning because it reduces parameters, memory usage and computational requirements while improving the ability to detect common features across an image.**



Q4. AlexNet vs ResNet

AlexNet and **ResNet** are important CNN architectures that significantly influenced the development of modern computer vision.

AlexNet demonstrated the effectiveness of deep CNNs for large-scale image classification, while ResNet introduced **residual connections**, allowing networks to become much deeper without suffering severely from degradation and vanishing-gradient problems.

Comparison of AlexNet and ResNet

Feature	AlexNet	ResNet
Introduced	2012	2015
Depth	8 learned layers	Available in 18, 34, 50, 101, 152+ layers
Architecture	Sequential CNN	Residual/block-based CNN
Convolutional filters	Larger early filters, such as 11×11	Mostly smaller 3×3 filters in modern variants
Parameter requirement	About 60 million	Depends on version; ResNet-50 has about 25.6 million
Feature extraction	Good hierarchical features	More powerful deep hierarchical features
Training difficulty	Easier than very deep networks	Residual connections make deep training easier
Vanishing gradient	More susceptible as depth increases	Significantly reduced using skip connections
Computational cost	High for its era	Higher for deeper variants

Feature	AlexNet	ResNet
Performance	Strong historical performance	Generally superior for complex recognition tasks
Main innovation	Deep CNN + ReLU + GPU training	Residual/skip connections

AlexNet

AlexNet contains convolutional layers followed by fully connected layers. Important features include:

- ReLU activation.
- Max pooling.
- Dropout for reducing overfitting.
- GPU-based training.
- Large convolutional filters in early layers.

AlexNet significantly improved image classification performance and demonstrated that deep CNNs could effectively learn image features.

ResNet

ResNet introduced **residual learning**.

Instead of directly learning:

$H(x)$

a residual block learns:

$$F(x) = H(x) - x$$

and produces:

$$H(x) = F(x) + x$$

The term x is passed through a shortcut connection.

Vanishing-Gradient Problem

As neural networks become deeper, gradients can become extremely small during backpropagation. This makes early layers difficult to train.

ResNet solves this problem more effectively through **skip connections**, which provide a direct path for information and gradients.

Feature Extraction

AlexNet learns features in a sequence of convolutional layers.

ResNet can contain many more layers, allowing it to learn highly complex hierarchical representations:

Edges → Textures → Shapes → Objects → Complex semantic features

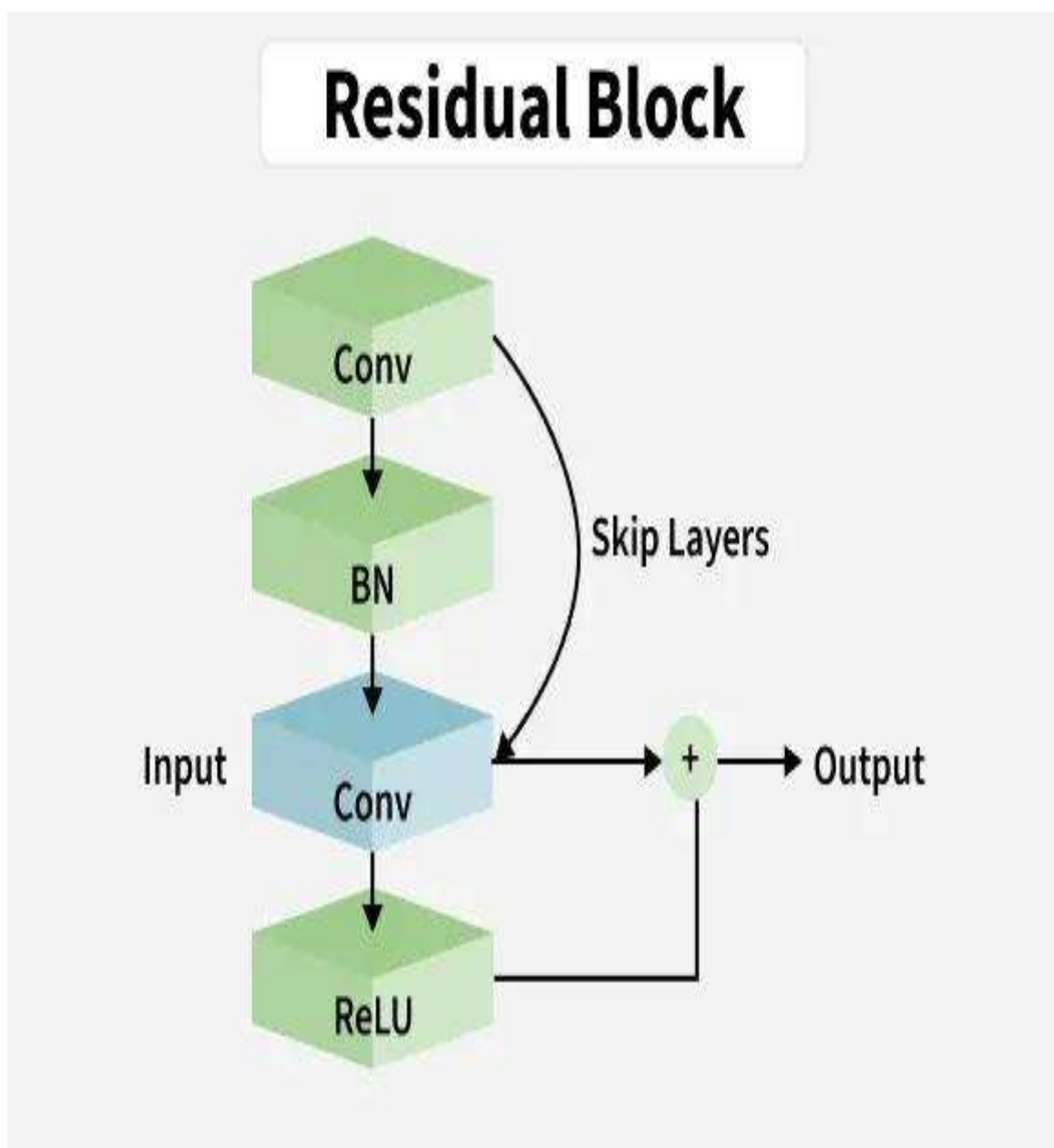
Recommendation

For a complex image recognition application, **ResNet is generally the better choice.**

The main reasons are:

1. It supports much deeper networks.
2. Skip connections reduce vanishing-gradient and degradation problems.
3. It provides stronger feature extraction.
4. It generally achieves better classification performance on complex image datasets.
5. Different ResNet variants provide flexibility between accuracy and computational cost.

Therefore, **ResNet is more suitable for complex modern image recognition applications, while AlexNet remains historically important as a pioneering deep CNN architecture.**



Q5. Regularized CNN vs Non-Regularized CNN

In plant disease classification, a CNN may achieve very high training accuracy but poor performance on unseen images. This is a common sign of **overfitting**.

Regularization techniques such as dropout, data augmentation and batch normalization can improve generalization and make the CNN more reliable.

Comparison

Feature	Non-Regularized CNN	Regularized CNN
Overfitting	High	Reduced
Training accuracy	May become extremely high	Usually slightly lower
Validation accuracy	Often poor	Usually better
Generalization	Poor	Better
Training stability	May be unstable	More stable
Memory/Computation	Lower	Slightly higher
Model robustness	Lower	Higher
Performance on unseen images	Poor	Better

1. CNN Without Regularization

A non-regularized CNN can memorize training images instead of learning general disease patterns.

For example, it may memorize:

- Specific leaf backgrounds.
- Lighting conditions.
- Leaf positions.
- Individual image characteristics.

As a result:

Training Accuracy = Very High

but

Testing Accuracy = Low

This indicates overfitting.

2. Dropout

Dropout randomly deactivates a percentage of neurons during training.

For example:

Dropout rate = 0.5

means approximately 50% of selected neurons are randomly ignored during a training step.

This prevents the network from becoming overly dependent on specific neurons and encourages it to learn more robust features.

3. Data Augmentation

Data augmentation artificially creates variations of existing training images.

Common operations include:

- Rotation
- Horizontal flipping
- Vertical flipping when appropriate
- Zooming
- Cropping
- Translation
- Brightness variation

For plant disease classification, augmentation is especially useful because leaves may appear under different orientations, lighting conditions and scales.

It effectively increases the diversity of the training dataset.

4. Batch Normalization

Batch normalization normalizes intermediate activations during training.

It helps:

- Stabilize training.
- Allow faster convergence.
- Reduce sensitivity to initialization.
- Improve training stability.

Batch normalization is primarily a **training-stabilization technique**, although it can also provide some regularizing effect.

Recommended Approach

For reliable plant disease classification, a combination of techniques is recommended:

Input Images → Data Augmentation → Convolution → ReLU → Batch Normalization → Pooling → Dropout → Fully Connected Layer → Softmax

The most important techniques for this problem are:

1. **Data Augmentation** – improves dataset diversity and generalization.
2. **Dropout** – reduces over-dependence on particular neurons.
3. **Batch Normalization** – improves training stability and convergence.

In addition, **early stopping** and appropriate weight decay can be considered if overfitting remains significant.

Conclusion

A non-regularized CNN can memorize training images and produce high training accuracy but poor performance on unseen leaf images. A regularized CNN learns more general disease characteristics and therefore performs better on real-world data.

Therefore, **data augmentation combined with dropout and batch normalization is a suitable approach for reliable plant disease classification**, with the exact strength of regularization selected using validation performance.

